OBINexus libpolycall v2 Restructure Plan

Feature Isolation Architecture with Isomorphic XML Mapping

🎯 Core Principles

1. **Feature Isolation**: Each feature is a self-contained module
2. **Isomorphic Mapping**: Direct 1:1 mapping between `polycall.xml` and C modules
3. **Schema Coherence**: XML configuration drives build targets
4. **Platform Independence**: Build targets for `.a`, `.so`, `.dll`

📁 New v2 Directory Structure

```
v2/
├── polycall.xml            # Master configuration (isomorphic mapping)
├── CMakeLists.txt            # Master build file
├── Makefile                # Feature-isolated make targets
│
├── include/
│   └── libpolycall/        # Public API headers
│       ├── core/           # Core functionality
│       │   ├── polycall.h      # Main public API
│       │   ├── context.h        # Context management
│       │   └── types.h         # Common types
│       │
│       ├── adapter/        # DOP Adapter System
│       │   ├── adapter.h        # Public adapter API
│       │   ├── ffi_bridge.h     # FFI bridge interface
│       │   └── types.h         # Adapter-specific types
│       │
│       ├── hotwire/        # Hot-swapping System
│       │   ├── hotwire.h        # Hot-reload public API
│       │   ├── router.h        # Routing interface
│       │   └── lifecycle.h     # Component lifecycle
│       │
│       ├── socket/         # Network Layer
│       │   ├── socket.h         # Socket public API
│       │   ├── protocol.h       # Protocol definitions
│       │   └── endpoint.h        # Endpoint management
│       │
│       ├── micro/          # Microservices
│       │   ├── service.h        # Service interface
│       │   ├── banking.h        # Banking service API
```

```
│   │   └── registry.h        # Service registry
│   │
│   ├── nlm/                  # NLM-Atlas Integration
│   │   ├── atlas.h           # NLM-Atlas API
│   │   ├── avl_huffman.h     # AVL-Huffman trie
│   │   └── namespace.h       # Namespace resolution
│   │
│   ├── stream/               # Event Streaming
│   │   ├── stream.h          # Stream API
│   │   ├── events.h          # Event definitions
│   │   └── channels.h        # Channel management
│   │
│   └── zero/                 # Zero-Trust Security
│       ├── trust.h           # Zero-trust API
│       ├── crypto.h          # Crypto primitives
│       └── token.h           # Token management
│
├── src/                      # Implementation files
│   ├── core/                 # Core implementation
│   │   ├── internal/         # Private headers
│   │   │   ├── core_private.h
│   │   │   └── state_machine.h
│   │   ├── polycall.c
│   │   ├── context.c
│   │   ├── tokenizer.c
│   │   └── CMakeLists.txt    # Feature-specific build
│   │
│   ├── adapter/              # DOP Adapter implementation
│   │   ├── internal/
│   │   │   ├── adapter_private.h
│   │   │   └── memory_pool.h
│   │   ├── adapter.c
│   │   ├── adapter_invoke.c
│   │   ├── adapter_lifecycle.c
│   │   ├── adapter_memory.c
│   │   ├── adapter_security.c
│   │   ├── adapter_utils.c
│   │   ├── bridge_registry.c
│   │   └── CMakeLists.txt
│   │
│   ├── hotwire/              # Hot-swapping implementation
│   │   ├── internal/
│   │   │   └── hotwire_private.h
│   │   ├── hotwire.c
│   │   ├── hotwire_router.c
│   │   ├── hotwire_loader.c
│   │   └── CMakeLists.txt
```

```
│   │
│   ├── socket/                 # Network implementation
│   │   ├── internal/
│   │   │   └── socket_private.h
│   │   ├── socket.c
│   │   ├── socket_config.c
│   │   ├── socket_endpoint.c
│   │   ├── socket_protocol.c
│   │   ├── socket_security.c
│   │   ├── socket_worker.c
│   │   └── CMakeLists.txt
│   │
│   ├── micro/                  # Microservice implementation
│   │   ├── internal/
│   │   │   └── service_private.h
│   │   ├── service_base.c
│   │   ├── banking_service.c
│   │   ├── auth_service.c
│   │   ├── document_service.c
│   │   ├── analytics_service.c
│   │   └── CMakeLists.txt
│   │
│   ├── nlm/                    # NLM-Atlas implementation
│   │   ├── internal/
│   │   │   └── nlm_private.h
│   │   ├── atlas.c
│   │   ├── avl_huffman.c
│   │   ├── namespace_resolver.c
│   │   └── CMakeLists.txt
│   │
│   ├── stream/                 # Stream implementation
│   │   ├── internal/
│   │   │   └── stream_private.h
│   │   ├── stream.c
│   │   ├── event_bus.c
│   │   ├── websocket.c
│   │   ├── sse.c
│   │   ├── grpc.c
│   │   └── CMakeLists.txt
│   │
│   └── zero/                   # Zero-trust implementation
│       ├── internal/
│       │   └── zero_private.h
│       ├── trust.c
│       ├── crypto_seed.c
│       ├── token_manager.c
│       ├── jwt.c
```

```
│       └── CMakeLists.txt
│
├── bindings/                   # Language bindings
│   ├── ffi/                    # Native FFI
│   │   ├── include/
│   │   │   └── polycall_ffi.h
│   │   ├── src/
│   │   │   ├── ffi_core.c
│   │   │   ├── ffi_adapter.c
│   │   │   └── ffi_micro.c
│   │   └── CMakeLists.txt
│   │
│   ├── python/                 # Python binding
│   │   ├── obinexus/
│   │   │   └── polycall/
│   │   └── setup.py
│   │
│   ├── node/                   # Node.js binding
│   │   ├── index.js
│   │   └── package.json
│   │
│   ├── rust/                   # Rust binding
│   │   ├── src/
│   │   └── Cargo.toml
│   │
│   └── go/                     # Go binding
│       ├── polycall.go
│       └── go.mod
│
├── config/                     # Configuration templates
│   ├── polycall.xml            # Master configuration
│   ├── features.xml            # Feature toggles
│   └── platform/
│       ├── linux.xml
│       ├── windows.xml
│       └── macos.xml
│
├── build/                      # Build output directory
│   ├── lib/
│   │   ├── static/             # Static libraries
│   │   │   ├── libpolycall_core.a
│   │   │   ├── libpolycall_adapter.a
│   │   │   ├── libpolycall_hotwire.a
│   │   │   ├── libpolycall_socket.a
│   │   │   ├── libpolycall_micro.a
│   │   │   ├── libpolycall_nlm.a
│   │   │   ├── libpolycall_stream.a
```

```
│   │   │   ├── libpolycall_zero.a
│   │   │   └── libpolycall.a       # Combined static library
│   │   │
│   │   └── shared/               # Shared libraries
│   │       ├── libpolycall_core.so
│   │       ├── libpolycall_adapter.so
│   │       ├── libpolycall_hotwire.so
│   │       ├── libpolycall_socket.so
│   │       ├── libpolycall_micro.so
│   │       ├── libpolycall_nlm.so
│   │       ├── libpolycall_stream.so
│   │       ├── libpolycall_zero.so
│   │       └── libpolycall.so.2.0.0  # Combined shared library
│   │
│   └── obj/                    # Object files (per feature)
│       ├── core/
│       ├── adapter/
│       ├── hotwire/
│       ├── socket/
│       ├── micro/
│       ├── nlm/
│       ├── stream/
│       └── zero/
│
├── scripts/                    # Build and deployment scripts
│   ├── configure.py            # Configuration generator
│   ├── build_feature.sh        # Feature-specific build
│   └── xml_to_cmake.py         # XML to CMake converter
│
└── test/                       # Test suite
    ├── unit/                   # Unit tests per feature
    │   ├── core/
    │   ├── adapter/
    │   ├── hotwire/
    │   ├── socket/
    │   ├── micro/
    │   ├── nlm/
    │   ├── stream/
    │   └── zero/
    │
    └── integration/            # Integration tests
        └── scenarios/
```

🔄 Isomorphic XML-to-C Mapping

polycall.xml → C Module Mapping

```xml
<!-- polycall.xml snippet -->
<adapter:mapping language="rust">
  <adapter:module>obinexus_polycall</adapter:module>
  <adapter:ffi-lib>libpolycall_rust.so</adapter:ffi-lib>
</adapter:mapping>
```

Maps to:

```c
// include/libpolycall/adapter/adapter.h
typedef struct polycall_adapter {
  const char* language;
  const char* module;
  const char* ffi_lib;
  // ... adapter methods
} polycall_adapter_t;

// src/adapter/adapter.c
polycall_adapter_t* polycall_adapter_load_rust(void) {
  // Implementation from XML config
}
```

Feature Activation via XML

```xml
```

```xml
<!-- config/features.xml -->
<features>
  <feature name="adapter" enabled="true">
    <modules>
      <module>adapter</module>
      <module>adapter_invoke</module>
      <module>adapter_lifecycle</module>
      <module>adapter_memory</module>
      <module>adapter_security</module>
    </modules>
    <dependencies>
      <dependency>core</dependency>
    </dependencies>
  </feature>

  <feature name="hotwire" enabled="true">
    <modules>
      <module>hotwire</module>
      <module>hotwire_router</module>
      <module>hotwire_loader</module>
    </modules>
    <dependencies>
      <dependency>core</dependency>
      <dependency>adapter</dependency>
    </dependencies>
  </feature>

  <feature name="nlm" enabled="true">
    <modules>
      <module>atlas</module>
      <module>avl_huffman</module>
      <module>namespace_resolver</module>
    </modules>
    <dependencies>
      <dependency>core</dependency>
      <dependency>socket</dependency>
    </dependencies>
  </feature>
</features>
```

🔨 Build System Integration

Master Makefile

```makefile
# v2/Makefile
include config/features.xml

# Feature-specific targets
FEATURES = core adapter hotwire socket micro nlm stream zero

# Build individual features
$(FEATURES):
  @echo "Building feature: $@"
  @$(MAKE) -C src/$@ CONFIG=$(PWD)/polycall.xml

# Build all enabled features
all: $(FEATURES)
  @echo "Linking libpolycall..."
  @scripts/link_features.sh

# Install headers with proper structure
install-headers:
  @for feature in $(FEATURES); do \
    install -d $(DESTDIR)/usr/include/libpolycall/$$feature; \
    install -m 644 include/libpolycall/$$feature/*.h \
      $(DESTDIR)/usr/include/libpolycall/$$feature/; \
  done

# Platform-specific builds
linux: CONFIG=config/platform/linux.xml
linux: all

windows: CONFIG=config/platform/windows.xml
windows: all

macos: CONFIG=config/platform/macos.xml
macos: all
```

Feature-Specific CMakeLists.txt

```cmake
```

```
# src/adapter/CMakeLists.txt
cmake_minimum_required(VERSION 3.10)
project(libpolycall_adapter)

# Read configuration from XML
include(${CMAKE_SOURCE_DIR}/scripts/xml_parser.cmake)
parse_xml_config(${CMAKE_SOURCE_DIR}/polycall.xml)

# Feature sources
set(ADAPTER_SOURCES
    adapter.c
    adapter_invoke.c
    adapter_lifecycle.c
    adapter_memory.c
    adapter_security.c
    adapter_utils.c
    bridge_registry.c
)

# Build static library
add_library(polycall_adapter_static STATIC ${ADAPTER_SOURCES})
target_include_directories(polycall_adapter_static
    PUBLIC ${CMAKE_SOURCE_DIR}/include
    PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/internal
)

# Build shared library
add_library(polycall_adapter_shared SHARED ${ADAPTER_SOURCES})
set_target_properties(polycall_adapter_shared PROPERTIES
    OUTPUT_NAME polycall_adapter
    VERSION 2.0.0
    SOVERSION 2
)

# Install targets
install(TARGETS polycall_adapter_static polycall_adapter_shared
    LIBRARY DESTINATION lib
    ARCHIVE DESTINATION lib
)
```

🎯 Usage Examples

Public API Usage

```c
// Using core feature
#include <libpolycall/core/polycall.h>

// Using specific features
#include <libpolycall/adapter/adapter.h>
#include <libpolycall/hotwire/hotwire.h>
#include <libpolycall/nlm/atlas.h>

int main() {
    // Initialize core
    polycall_context_t* ctx = polycall_init("polycall.xml");

    // Load adapter
    polycall_adapter_t* adapter = polycall_adapter_load(ctx, "rust");

    // Enable hot-swapping
    polycall_hotwire_enable(ctx, adapter);

    // Use NLM-Atlas for service discovery
    nlm_atlas_service_t* service = nlm_atlas_discover(ctx,
        "debit.validate.obinexus.banking.finance.us.org");

    return 0;
}
```

Feature Isolation Build

```bash
# Build only core feature
make core

# Build adapter with dependencies
make adapter  # Automatically builds core first

# Build specific platform
make linux CONFIG=config/platform/linux.xml

# Build with feature flags
make FEATURES="core adapter hotwire" all
```

📋 Schema Validation Script

```python
```

```python
#!/usr/bin/env python3
# scripts/validate_schema.py

import xml.etree.ElementTree as ET
import os

def validate_isomorphic_mapping(xml_file, src_dir):
    """Validate XML configuration matches C implementation"""
    tree = ET.parse(xml_file)
    root = tree.getroot()

    errors = []

    # Check adapter mappings
    for adapter in root.findall('.//adapter:mapping',
                                {'adapter': 'http://obinexus.com/schema/adapter'}):
        language = adapter.get('language')
        module = adapter.find('adapter:module').text
        ffi_lib = adapter.find('adapter:ffi-lib').text

        # Check if corresponding C file exists
        c_file = f"{src_dir}/adapter/adapter_{language}.c"
        if not os.path.exists(c_file):
            errors.append(f"Missing implementation for {language} adapter")

    # Check service mappings
    for service in root.findall('.//web:service',
                                {'web': 'http://obinexus.com/schema/web'}):
        service_id = service.get('id')
        c_file = f"{src_dir}/micro/{service_id}_service.c"
        if not os.path.exists(c_file):
            errors.append(f"Missing implementation for {service_id} service")

    return errors

if __name__ == "__main__":
    errors = validate_isomorphic_mapping("polycall.xml", "src")
    if errors:
        print("❌ Schema validation failed:")
        for error in errors:
            print(f"  - {error}")
    else:
        print("✅ Schema validation passed")
```

🚀 Implementation Steps

Phase 1: Directory Restructure

```bash
# Create new structure
cd v2/
mkdir -p include/libpolycall/{core,adapter,hotwire,socket,micro,nlm,stream,zero}
mkdir -p src/{core,adapter,hotwire,socket,micro,nlm,stream,zero}/internal

# Move existing files to new structure
mv src/core/dop/* src/adapter/
mv include/polycall*.h include/libpolycall/core/
```

Phase 2: Update Include Guards

```c
// Before: include/polycall.h
#ifndef POLYCALL_H
#define POLYCALL_H

// After: include/libpolycall/core/polycall.h
#ifndef LIBPOLYCALL_CORE_POLYCALL_H
#define LIBPOLYCALL_CORE_POLYCALL_H
```

Phase 3: Create Feature CMakeLists

```bash
# Generate CMakeLists for each feature
for feature in core adapter hotwire socket micro nlm stream zero; do
    scripts/generate_cmake.sh $feature > src/$feature/CMakeLists.txt
done
```

Phase 4: XML Configuration Generator

```bash
# Generate configuration from existing code
python scripts/configure.py --scan src/ --output polycall.xml
```

✅ Success Metrics

- [ ] All includes use `#include <libpolycall/feature/header.h>` format
- [ ] Each feature builds independently: `make feature`
- [ ] XML configuration drives build process
- [ ] Platform-specific builds work: `make linux/windows/macos`
- [ ] All tests pass: `make test`
- [ ] Documentation generated from XML: `make docs`
- [ ] FFI bindings work with new structure
- [ ] Hot-swapping functional with isolated features

🔐 Security & Isolation

Compilation Isolation

- Each feature compiles to separate `.o` files
- Features link only declared dependencies
- Private headers in `internal/` directories
- Public API strictly controlled

Runtime Isolation

- Features loaded dynamically via `dlopen()` when needed
- Zero-trust verification between features
- Memory pools per feature
- Thread-safe feature activation/deactivation

Configuration Security

- XML schema validation before build
- Cryptographic signing of feature modules
- Runtime verification of module integrity
- Audit logging of feature activation

This restructure provides complete feature isolation with isomorphic XML-to-C mapping, ensuring clean separation of concerns and modular build capabilities.